Supply-Chain Knowledge Graph: Frozen V1 Architecture

Goal. Build a reproducible supply-chain KG from unstructured text. V1 focuses on data acquisition, LLM-based entity/relation extraction, entity disambiguation, controlled upstream expansion, and Neo4j storage. Risk analysis and existing-edge monitoring are outside V1.

1. Decisions on the four open questions

Question	Frozen V1 decision
Seed definition	Seeds are configurable and can be increased. The default replication config uses AlMahri's 13 selected companies: Toyota, Tesla, BMW, BYD; Samsung SDI, CATL, Panasonic; Zijin Mining, Albemarle, Tianqi Lithium, Ganfeng Lithium, Norilsk Nickel, Jinchuan Group.
Graph expansion	Node-centric and tier-aware, not pairwise. Each frontier company is searched with a small set of tier-specific query templates. Newly discovered upstream companies become the next frontier.
Boundary / stopping	`max_tier` is configurable. Following Prof. Kuo's suggestion for testing, the default is **3 upstream supply-chain tiers** from the final EV product: seeds are Tier 0, discovered suppliers are Tiers 1–3, and Tier 3 is terminal. Only supply-chain relations (mainly `SUPPLIES_TO`) advance the tier.
Ontology	Static in V1. We use the AlMahri evaluation schema: entity types `Company, Location, Material, Person, Product, Mine`; relations `suppliesTo, locatedIn, owns, produces`. A later mapping to SCRO can be added without changing the extraction pipeline.

2. AlMahri baseline, data modes, and datasets

AlMahri et al. created a controlled evaluation corpus of **1,277 Wikipedia sentences** and manually annotated it for six entity types and four relation types. Their method uses zero-shot LLM prompting for NER/RE and a second prompt for entity disambiguation. The exact annotated corpus was described in the paper, but we did not identify a public downloadable package for the full labeled dataset. Because that exact labeled package is not available to us, V1 is dataset-agnostic and exposes two operating modes: `INGEST` for a fixed corpus/dataset already in hand, and `EXPAND` when the system must discover documents from seeds.

Mode	Behavior
`INGEST`	The user provides documents or a dataset. The system normalizes, extracts facts, resolves duplicates, and merges them into Neo4j. No search is performed. This is the closest mode to the fixed-data evaluation setup in AlMahri.
`EXPAND`	The user provides seeds. For each frontier company, the system runs tier-specific queries over enabled retrieval sources (e.g., Wikipedia search and Google News / GDELT), extracts upstream suppliers from the returned documents, and continues until the configured tier boundary. Wikipedia is a source in the search layer; the system does not assume that the entity's own Wikipedia page is the document to retrieve.

Auxiliary reference dataset. We use **Sectivia** as an external graph sanity-check dataset: 350 directed supplier-to-customer relations across 214 listed companies (215 company rows in the node file), released as CSV/JSON under CC BY 4.0. It is useful for checking discovered links, but it is *not* a substitute for AlMahri's annotated text corpus.

3. V1 architecture

INGEST: Documents → Normalize/Store → LLM Prompt 1 → Prompt 2 → Facts → Neo4j MERGE

EXPAND: Seeds → Persistent Frontier → Tier-Specific Retrieval Queries → Wikipedia/Search/News Sources → Dedup/Filter → LLM Prompt 1 → Prompt 2 → Neo4j → New Upstream Companies → Next Tier

The implementation should reuse Neo4j GraphRAG KG Builder components for splitting, schema-guided entity/relation extraction, and graph writing where practical. Custom code is limited to source retrieval, expansion policy, checkpointing, provider adapters, and AlMahri-style disambiguation.

4. Graph expansion algorithm

1. Initialize a persistent queue with user seeds at Tier 0.
2. Pop one `Company` and load the query templates and retrieval-source settings for its tier.
3. Run each query independently over the enabled sources (e.g., Wikipedia search, Google News, GDELT). Do not generate pairwise company queries.
4. Merge returned documents/results, normalize URLs, remove duplicates, and keep at most the configured number of results *per query*. Fetch/filter only useful documents.
5. Extract facts. If a new company has a valid upstream supply relation to the current node, assign `tier=current+1`, merge it into the KG, and enqueue it only if `tier < max_tier`. Tier-3 nodes are therefore stored but not expanded under the default setting.
6. Continue until the frontier is empty or the tier/node limits are reached.

Query design. We never create all company pairs. Queries are **node-centric**: each tier has a configurable set of keywords/query templates, and the current frontier company's name is inserted into them. Search cost is approximately $O(NK)$, where N is the number of expanded companies and K is the small number of templates for that tier, rather than $O(N^2)$.
Example. If BMW is a Tier-0 frontier node, Tier-0 templates such as `"{entity}" suppliers`, `"{entity}" sourcing`, and `"{entity}" battery suppliers` become `"BMW" suppliers`, `"BMW" sourcing`, and `"BMW" battery suppliers`. If CATL is then discovered as an upstream supplier, it becomes a Tier-1 frontier node and is searched with the Tier-1 set, e.g., `"CATL" suppliers`, `"CATL" component suppliers`, and `"CATL" raw material suppliers`. Tier-2 templates can focus further upstream, e.g., `"{entity}" material suppliers` and `"{entity}" mines`. Each template can have its own result limit through the same config hierarchy described below.

Config inheritance. Search limits use "closest value wins":

$$\text{global defaults} < \text{tier defaults} < \text{query override}.$$

For example, the global default may be 20 results/query, Tier 0 may override it to 30, and one specific query may override it again to 15. This rule is applied per field, so only explicitly changed values are replaced.

5. LLM prompts and extraction

Prompt	V1 behavior
Prompt 1: NER + RE	**Purpose: extract graph facts from text.** The prompt receives a document/chunk plus the fixed ontology and asks the model to return only allowed entity types and relations. Its output is structured entities and triples, together with the supporting text/source evidence. For example, from "CATL supplies battery cells to BMW", it should identify CATL and BMW as `Company` entities and produce `CATL suppliesTo BMW`. NER and relation extraction are conceptually separate tasks, but V1 can obtain both in one structured LLM call.
Prompt 2: Entity disambiguation	**Purpose: merge mentions that refer to the same real-world entity.** After Prompt 1, mentions are grouped by entity type and passed to this second prompt. The model assigns the same canonical/group ID to equivalent mentions, e.g., "CATL" and "Contemporary Amperex Technology Co. Limited", while keeping genuinely different entities separate. The resolved IDs are then used before Neo4j `MERGE`, preventing duplicate nodes.

Prompt flow: document text + static ontology → **Prompt 1** → extracted entities/relations/evidence → **Prompt 2** → canonical entity groups → deduplicated facts → Neo4j.

The LLM layer is provider-independent. The planned local model is **Qwen3.8-27B** in instruct/non-thinking mode, served with vLLM; a GPT API can be kept as an optional comparison/provider. The same extraction contract is used in either case, so the KG logic does not depend on the model backend.

6. Reliability and scope

- **Resume is default.** Both modes save state in a small persistent store (e.g., SQLite): document status for `INGEST`, and frontier/search/document status for `EXPAND`. A user can explicitly request a fresh run.
- **No duplicate work.** Raw documents are stored with normalized URL/content hash; normalized data and extracted facts are saved separately (JSONL/Parquet). Completed LLM results are reused after restart.
- **Idempotent graph writes.** Canonical entity IDs plus Neo4j `MERGE` prevent duplicate nodes/edges when a run is resumed or repeated.
- **Not in V1.** Existing-edge monitoring, relationship-change detection, risk analytics, and custom graph dashboards. Neo4j Browser is sufficient for inspection/visualization.

References. [1] S. AlMahri, L. Xu, A. Brintrup, "Enhancing Supply Chain Visibility with Knowledge Graphs and Large Language Models," IJPR / arXiv:2408.07705. [2] Neo4j GraphRAG Python, Knowledge Graph Builder documentation. [3] IOFoundry, Supply Chain Reference Ontology (SCRO). [4] Sectivia, Open Supply-Chain Dataset (CC BY 4.0).